

Fractal Computing

The HERMES Runtime Model Primitives, Operators, Execution Architecture, and Implications

A Formal Specification of Operator-Based Fractal Reasoning Systems

Author: Matthew B Wareham

Date: July 2026

Version: 2.0

Classification: Formal Technical Specification

This document constitutes the formal specification of the HERMES Runtime, versions 1.0 and 2.0. All operator definitions, world-model schemas, cycle engine descriptions, and meta-logic constraints presented herein are normative unless explicitly noted as informative.

Abstract

We introduce **fractal computing** as a formal computational paradigm in which reasoning, transformation, and synthesis processes exhibit strict self-similar structure across multiple scales of execution. In a fractal computing system, the same structural form that governs a single reasoning step also governs an entire reasoning session — the micro-cycle and macro-cycle are isomorphic. This scale-invariant property distinguishes fractal computing from classical von Neumann architectures and from symbolic AI systems, neither of which possesses native mechanisms for recursive self-modeling or cross-scale

structural coherence.

We present **HERMES** (the meta-logic and cognitive magnifier runtime) as a concrete, formally specified implementation of the fractal computing paradigm. HERMES is defined by three foundational primitives — the World-Model, the Identity Model, and the Narrative State — and by a suite of twelve operators: GENERATE, UNFOLD, COLLAPSE, INTEGRATE, COMPLETE, RETURN, INVERT, PERTURB, HARMONIZE, IDENTITY_STABILIZE, RECURSE, and CONTINUITY_ENFORCE. These operators constitute a complete computational basis for cognitive transformation.

The HERMES cycle engine coordinates operator execution across two scales: a six-stage macro-cycle governing global session-level reasoning, and a four-stage micro-cycle governing step-level reasoning within each macro-stage. Eight meta-logic constraints — extended from five in version 1.0 to eight in version 2.0 — serve as runtime invariants guaranteeing coherence, continuity, and bounded execution.

Version 2.0 of the HERMES specification introduces significant enhancements: entropy-guided execution through new Entropy and Coherence Gradient fields in the world-model schema; a three-layer identity architecture distinguishing Core, Narrative, and Situational identity; expanded operator semantics including entropy-aware collapse selection, tri-resolution harmonization, and entropy-capped recursion; and a ten-step reasoning protocol replacing the seven-step protocol of version 1.0. We discuss theoretical implications, connections to established computational models, and application domains ranging from meta-cognitive AI agents to autonomous systems and cognitive science.

Table of Contents

Abstract	2
Section 1 — Introduction	4

Section 2 — Foundational Primitives	5
2.1 The World-Model	5
2.2 The Identity Model	7
2.3 The Narrative State	8
Section 3 — The Operator Suite	9
3.1–3.12 All Twelve Operators	9
Section 4 — The Cycle Engine	22
4.1 The Full Macro-Cycle	22
4.2 The Micro-Cycle (v2)	23
4.3 Fractal Self-Similarity	23
4.4 Cycle Efficiency (v2 Upgrades)	24
Section 5 — Meta-Logic Constraints	25
Section 6 — The Reasoning Protocol	28
Section 7 — Runtime Architecture and Implementation	30
Section 8 — Fractal Properties and Theoretical Implications	32
Section 9 — Applications and Implications	34
Section 10 — Version History: v1.0 to v2.0	36
Section 11 — Conclusion	37
Glossary of Terms	38

1. Introduction

1.1 What Is Fractal Computing?

We define **fractal computing** as a computational paradigm in which reasoning, transformation, and synthesis processes exhibit strict self-similar structure across multiple scales of operation. A fractal computing system is one in which the same structural form — the same sequence of generative, expansive, reductive, integrative, and completive stages — governs both the smallest unit of reasoning

and the largest session-level arc of computation. The micro-scale and the macro-scale are not merely analogous; they are isomorphic.

This property — self-similarity across scales — is borrowed conceptually from fractal geometry, where a structure such as the Mandelbrot set or a coastline exhibits detail of the same qualitative character regardless of the magnification at which it is examined. In fractal computing, we claim a precisely analogous property: reasoning quality, coherence, and structural integrity are maintained regardless of the recursion depth at which analysis is conducted.

Key Concept: Self-Similarity

In fractal computing, the structure of a single reasoning step — generate, expand, collapse, integrate, complete, return — is identical in form to the structure of an entire reasoning session. The system contains itself at every scale.

1.2 Contrast With Classical Computational Models

Classical **von Neumann architectures** separate computation into a stored-program memory model with a fetch-decode-execute cycle. The execution cycle is uniform and flat: each instruction cycle is structurally independent, with no intrinsic mechanism for a step to reason about its own execution, to model its own state trajectory, or to adjust its own cycle structure based on the content of its outputs. Such architectures are extraordinarily powerful for deterministic, well-specified computation, but they provide no native support for meta-cognition, narrative continuity, or identity coherence.

Symbolic AI systems — production systems, expert systems, logic programming environments — introduce the capacity for declarative knowledge representation and deductive inference. They can, in principle, represent facts about their own knowledge state. However, they lack a general-purpose mechanism for structured self-application at arbitrary depth, for maintaining coherent identity across recursive self-modification, and for resolving contradiction without execution halting. The classical resolution: halt and report failure, or apply a fixed conflict-resolution strategy that does not adapt to the semantic content of the conflict.

Fractal computing addresses these limitations directly. The system's execution structure is itself a knowledge structure. Reasoning about one's own reasoning is not a special case handled by an external meta-interpreter; it is an ordinary application of the same operators to the world-model's representation of the current execution state.

1.3 Introducing HERMES

We introduce **HERMES** — the meta-logic and cognitive magnifier runtime — as a concrete, formally specified implementation of the fractal computing paradigm.

HERMES is defined by:

- Three **foundational primitives**: the World-Model, the Identity Model, and the Narrative State
- Twelve **operators**: formally specified transformation functions that constitute a complete basis for cognitive computation
- A **cycle engine**: the fractal heartbeat of the system, coordinating operator sequences across macro- and micro-scale cycles
- Eight **meta-logic constraints**: runtime invariants that bound execution and prevent degenerate states
- A **reasoning protocol**: a ten-step execution sequence instantiating one complete macro-cycle

1.4 Motivation

The core motivation for HERMES is the construction of cognitive systems capable of four properties that classical architectures do not support natively:

1. **Meta-cognition**: the ability to reason about one's own reasoning process, to observe the current state of one's own world-model, and to modify the execution strategy accordingly.
2. **Identity coherence across recursion**: the ability to maintain a stable and consistent self-model even as the system applies its reasoning operators to its own outputs at arbitrary depth.

3. **Contradiction resolution without halting:** the ability to detect, process, and resolve contradictory states through operator application rather than by raising an exception or terminating.
4. **Narrative continuity:** the ability to maintain a coherent, traceable history of reasoning transitions across the full arc of a session, enabling retrospective audit and forward-looking anticipation.

1.5 Scope of This Document

This document constitutes the normative formal specification of the HERMES Runtime, versions 1.0 and 2.0. We present: the complete world-model schema; the three-layer identity architecture; the narrative state structure; the full specification of all twelve operators including input/output types, transformation rules, failure modes, invariants, and version 2.0 upgrades; the cycle engine at both macro- and micro-scale; all eight meta-logic constraints; the ten-step reasoning protocol; and a discussion of runtime architecture, theoretical implications, and application domains.

2. Foundational Primitives

The HERMES runtime is grounded in three foundational primitives. These are not derived structures — they are the irreducible representational substrate upon which all operator computation is defined. Every operator takes at least one of these primitives as input or produces at least one as output. Together, they constitute the complete state of a HERMES execution instance at any point in time.

2.1 The World-Model

The World-Model is the central, persistent knowledge representation of the HERMES runtime. It is a structured, mutable graph of nodes and relations encoding everything the system currently knows — explicitly, implicitly, and potentially. The World-Model is the substrate that operators read from and write to; it is both the input and the output of the macro-cycle considered as a whole. Its schema is defined as follows:

Field	Version	Description
CONCEPTS	v1, v2	Explicit conceptual nodes. Named, typed, and attributed knowledge units forming the semantic vocabulary of the current reasoning context.
ENTITIES	v1, v2	Objects with identity and attributes. Distinguished from concepts by having persistent identity across state transitions. Entities are the primary bearers of relational predicates.
CONTEXTS	v1, v2	Situational frames bounding the interpretation of concepts and entities. Each context defines a scope within which relations and gradients are evaluated.
GRADIENTS (v1)	v1	Relevance, Intensity, and Certainty — scalar scores attached to nodes and relations quantifying their current salience, activation strength, and epistemic status.
GRADIENTS (v2)	v2	Extended with Coherence Gradient (measuring internal consistency of the model) and Entropy Gradient (measuring informational disorder and unpredictability within the current state).
RELATIONS (v1)	v1	Typed edges between nodes: CAUSE, CONTRAST, SUPPORT, DEPENDENCE, CONFLICT.
RELATIONS (v2)	v2	Extended with Emergent relations (relations inferred from repeated co-occurrence patterns) and Meta-relations (relations between relations, enabling higher-order structural modeling).
IDENTITY (v1)	v1	Represented as three fields: AGENTS (active reasoning participants), PERSPECTIVES (viewpoints held by agents), ROLES (functional positions within the current context).
IDENTITY (v2)	v2	Replaced by the three-layer identity model: Core identity, Narrative identity, Situational identity. See Section 2.2.
STATES (v1)	v1	CURRENT_STATE, HISTORICAL_STATES, POTENTIAL_STATES.
STATES (v2)	v2	Split into Micro-state (step-level, transient, high-resolution) and Macro-state (session-level, persistent, low-resolution summary). Historical and potential state collections retained.
TEMPORAL_WEIGHTS	v2	Three scalar weights applied to historical states: Recency (how recently the state occurred), Relevance (how applicable it is to the current context), Persistence (how stable it has been across multiple

Field	Version	Description
		cycles).
LATENT_STRUCTURES	v2	Implicit patterns inferred from repeated state configurations. These structures are not asserted explicitly but surface through statistical regularity across the historical state log. They represent the system's accumulated implicit knowledge.
NARRATIVE_NODES	v1, v2	Structured narrative elements: EVENTS (discrete occurrences), TRANSITIONS (state change records), GOALS (active objectives), OUTCOMES (resolved results of goals and events).
RECURSION_DEPTH	v1	Integer counter in range 0–32, mandatory tracking. Must be incremented by any operator invoking self-referential or iterative reasoning. Exit condition or depth cap required.
RECURSION_DEPTH	v2	Dynamic maximum replaces fixed cap of 32. Maximum is computed as a function of the current Entropy Gradient — higher entropy allows shallower recursion before automatic cap enforcement.

Normative Note

The World-Model is not a static data structure. It is a living substrate

— it is read, written, and updated by every operator execution. Operators that do not explicitly modify the World-Model still implicitly depend on its current state for input resolution. No operator may assume a stale or previously cached World-Model unless explicitly operating in a sandboxed recursion context with documented isolation.

2.2 The Identity Model

Identity in the HERMES runtime is not a simple label or tag. In version 1.0, identity was represented as a flat triple of AGENTS, PERSPECTIVES, and ROLES — sufficient for

shallow reasoning contexts but inadequate for systems that apply reasoning operators to their own outputs at arbitrary depth. Version 2.0 introduces the **three-layer identity architecture**, which provides a principled and formally grounded account of identity coherence across recursive self-application.

The three layers are as follows:

Layer	Name	Description	Persistence
Layer 1	Core Identity	Persistent fundamental traits, values, and invariant properties of the reasoning agent. These are explicitly protected: only the IDENTITY_STABILIZE operator with explicit override authorization may modify core identity fields. Core identity is the stable anchor of selfhood.	Permanent unless explicitly modified
Layer 2	Narrative Identity	The evolving story of the reasoning agent as accumulated across reasoning sessions. It integrates events, transitions, outcomes, and role changes into a coherent autobiographical arc. Narrative identity is updated at cycle boundaries by RETURN and CONTINUITY_ENFORCE.	Persistent ; updated at cycle boundaries
Layer 3	Situational Identity	Context-specific role, stance, and operative perspective adopted within a particular reasoning context or task. Situational identity is the most mutable layer and may be updated by PERTURB, HARMONIZE, and INTEGRATE without triggering identity fragmentation, provided core and narrative identity remain coherent.	Context-local; updated per cycle stage

Identity coherence is not merely a desideratum — it is a formal requirement. In a fractal system, the reasoning agent applies its operators to its own outputs. If the agent's self-model is internally inconsistent, this self-application will produce incoherent results that propagate and amplify through successive recursion cycles. Without a principled identity architecture, recursive self-application collapses into incoherence within a bounded number of cycles. The three-layer model provides the structural separation necessary to prevent this: core identity remains stable even as situational identity adapts freely, and narrative identity provides the bridging continuity that makes the stability of core identity interpretable in context.

Why Identity Coherence Matters in Fractal Systems

Consider a recursive system that applies operator

R

to its own output:

$R(R(R(\dots)))$

. If the system's self-model shifts between applications, each invocation of

R

operates on a subtly different understanding of what the system is doing and why.

Over sufficient depth, these small shifts accumulate into catastrophic incoherence.

The three-layer identity model eliminates this failure mode by ensuring that Core Identity — the system's most fundamental self-understanding — remains invariant across all recursion depths.

2.3 The Narrative State

The Narrative State is the second foundational primitive, and it serves a purpose that is distinct from but complementary to the World-Model. Where the World-Model is the system's representation of *what is known*, the Narrative State is the system's representation of *how it came to know it* — the execution trace, the story of transitions, the record of decisions made and outcomes achieved.

The Narrative State is not merely a log. It is an active computational structure that operators read from and write to. It enables:

- **Continuity across recursion boundaries:** when a RECURSE operator increases the recursion depth, the Narrative State at the new depth is initialized from the current Narrative State, ensuring that the recursive sub-cycle has access to the full history of the parent cycle.
- **Forward anticipation:** by examining the Narrative State's GOALS and POTENTIAL_STATES, the cycle engine can predict which operators are most

likely to be needed in upcoming stages and pre-position resources accordingly (v2).

- **Retrospective audit:** the complete Narrative State is preserved in `HISTORICAL_STATES` across cycles, enabling after-the-fact analysis of how a particular conclusion was reached.
- **Gap detection:** the `CONTINUITY_ENFORCE` operator (v2) scans the Narrative State for discontinuities — missing transitions, unresolved goals, orphaned events — and either patches them automatically or raises a constraint violation.

The Narrative State's primary fields are: `EVENTS`, `TRANSITIONS`, `GOALS`, and `OUTCOMES`. In version 2.0, these are supplemented by the Narrative Identity layer of the Identity Model, which provides the interpretive frame within which the narrative's events are understood as meaningful rather than as mere state deltas.

3. The Operator Suite

Operators are the computational primitives of the HERMES runtime. They are formally specified transformation functions: each operator takes typed inputs, applies a deterministic (or semi-deterministic, for operators involving exploration) transformation rule, and produces typed outputs. Operators may also modify the World-Model and Narrative State as side effects, where specified.

HERMES defines exactly twelve operators. These twelve constitute a **complete computational basis** for the fractal reasoning paradigm: every cognitive transformation required by the system can be expressed as a finite sequence of applications of these operators. No additional operators are defined or permitted in the normative specification.

Each operator is presented below in a consistent formal block structure, including: Purpose, Input Types, Output Types, Transformation Rule, Failure Modes, Invariants, and v2.0 upgrades where applicable.

3.1 GENERATE

GENERATE

v1

Purpose	Instantiate the initial World-Model structures, entities, concepts, and relations from a raw input prompt. GENERATE is the entry point of every macro-cycle; it converts an unstructured or semi-structured input into a populated World-Model ready for expansion.
Input Types	PROMPT, WORLD_MODEL
Output Types	WORLD_MODEL, NARRATIVE_STATE
Transformation Rule	Parse PROMPT to extract concepts, entities, relations, contexts, and initial gradient values. Instantiate these structures within the WORLD_MODEL. Initialize NARRATIVE_STATE with a root EVENT corresponding to the invocation of GENERATE. Set RECURSION_DEPTH = 0 if not already set. Assign initial Relevance, Intensity, and Certainty gradients based on PROMPT salience analysis.
Failure Modes	UNDER_SPECIFICATION: PROMPT does not contain sufficient information to instantiate a coherent initial WORLD_MODEL. INCOHERENT_INITIAL_STATE: The structures inferred from PROMPT are internally contradictory at instantiation time.
Invariants	STRUCTURAL_VALIDITY: The output WORLD_MODEL must be a well-formed graph satisfying schema constraints. All mandatory fields must be populated. RECURSION_DEPTH must be set.

3.2 UNFOLD

UNFOLD

v1

Purpose	Expand the current World-Model by deriving implications, consequences, and developments from the existing state. UNFOLD is the expansive phase of the cycle — it increases the information density of the World-Model by following causal and logical chains forward.
Input Types	WORLD_MODEL, NARRATIVE_STATE
Output Types	WORLD_MODEL, NARRATIVE_STATE
Transformation Rule	For each active CONCEPT, ENTITY, and RELATION in WORLD_MODEL, trace CAUSE and DEPENDENCE relations forward to derive implied consequences. Add derived nodes and edges to WORLD_MODEL. Update NARRATIVE_STATE with TRANSITION records for each significant expansion step. Increment gradient values for newly activated nodes based on propagation weight of their source nodes.
Failure Modes	EXPANSION_INCOHERENT: Forward inference produces a structural contradiction that cannot be represented consistently within the current WORLD_MODEL schema. STATE_OVERLOAD: Expansion produces so many new nodes and edges that the WORLD_MODEL exceeds representational capacity or coherence threshold degrades below minimum.
Invariants	CONSISTENT_CAUSALITY: All derived CAUSE relations must be transitive-consistent with existing CAUSE relations. IDENTITY_TRACKING: Entity identity must be preserved across expansion; no entity may be silently split or merged during UNFOLD.

3.3 COLLAPSE

COLLAPSE

v1

v2 upgraded

Purpose	Reduce the complexity of the expanded World-Model by selecting the most salient threads, structures, and resolutions while pruning low-relevance branches. COLLAPSE is the reductive phase of the cycle — it narrows the World-Model from its maximum expansion to a focused, tractable state.
Input Types	WORLD_MODEL, NARRATIVE_STATE
Output Types	FOCUSED_STATE
Transformation Rule	Score all nodes and edges in WORLD_MODEL by combined Relevance and Intensity gradient values. Select the highest-scoring connected subgraph that satisfies the minimum coherence threshold. Prune all nodes and edges below selection threshold. Record pruned elements in NARRATIVE_STATE as historical data (not discarded permanently). Produce FOCUSED_STATE as the selected subgraph with updated gradient values.
Failure Modes	LOSS_OF_CRITICAL_INFORMATION: Pruning removes nodes that carry constraint dependencies required by active meta-logic invariants. ARBITRARY_SELECTION: The scoring function cannot distinguish between candidate subgraphs, producing a non-deterministic selection without documented tie-breaking rationale.
Invariants	RELEVANCE_MAXIMIZATION: The selected FOCUSED_STATE must have maximal aggregate Relevance gradient among all valid subgraphs of the same cardinality. TRACEABILITY: All pruned elements must be traceable via NARRATIVE_STATE historical records.
v2 Upgrade	Entropy-aware collapse selection: In v2, the Entropy Gradient of the WORLD_MODEL is used as a second-order weighting factor. High-entropy regions of the WORLD_MODEL are preferentially collapsed more aggressively; low-entropy (high-certainty) regions are preferentially retained. This prevents the loss of stable, well-established knowledge during collapse of a high-disorder state.

3.4 INTEGRATE

INTEGRATE

v1

v2 upgraded

Purpose	Merge the elements of the FOCUSED_STATE with any unresolved conflicts into a unified, internally stable configuration. INTEGRATE is the synthesis phase — it produces from the collapsed and selected elements a single coherent state ready for completion.
Input Types	FOCUSED_STATE, CONFLICT_SET
Output Types	COHERENT_STATE
Transformation Rule	For each element in FOCUSED_STATE, check for CONFLICT relations with other elements. Apply conflict resolution policy (default: highest-Certainty element takes precedence; ties resolved by Persistence weight). Merge resolved elements into a unified COHERENT_STATE. Update CONFLICT_SET to reflect resolved and residual conflicts. Record resolution decisions in NARRATIVE_STATE as TRANSITION entries.
Failure Modes	INTEGRATION_FAILURE: A conflict exists between elements of FOCUSED_STATE that cannot be resolved by any available policy without violating a meta-logic constraint. RESIDUAL_CONFLICT: After integration, one or more unresolved conflicts remain in CONFLICT_SET at levels above tolerance threshold.
Invariants	COHERENCE: The output COHERENT_STATE must satisfy the minimum Coherence Gradient threshold. IDENTITY_STABILITY: The integration process must not alter Core Identity or Narrative Identity fields without explicit operator authorization.
v2 Upgrade	Coherence-maximizing integration with world-model update: In v2, the integration resolution policy is replaced by a coherence-maximizing selection that explicitly evaluates multiple candidate integrations and selects the one that maximizes the Coherence Gradient of the resulting COHERENT_STATE. The selected integration is then used to update the corresponding fields in the WORLD_MODEL directly.

3.5 COMPLETE

COMPLETE
v1

Purpose	Mark a reasoning or narrative cycle as structurally complete. COMPLETE performs the formal closure operation: it validates that the COHERENT_STATE satisfies all active meta-logic constraints and that no pending violations remain before the cycle is sealed.
Input Types	COHERENT_STATE
Output Types	FINAL_STATE
Transformation Rule	Evaluate COHERENT_STATE against all active meta-logic constraints. If all constraints pass: produce FINAL_STATE as a sealed, immutable snapshot of COHERENT_STATE; append a COMPLETION OUTCOME to NARRATIVE_STATE. If any constraint fails: return constraint violation report; do not produce FINAL_STATE; trigger AUTO-PATCH or escalate to HARMONIZE depending on violation severity.
Failure Modes	PREMATURE_COMPLETION: COMPLETE invoked before COLLAPSE and INTEGRATE have been applied in the current cycle (violates CYCLE_COMPLETION_REQUIREMENT). UNRESOLVED_ELEMENTS: Active GOALS in NARRATIVE_STATE have not been resolved; active CONFLICT entries remain above threshold.
Invariants	NO_PENDING_CONSTRAINT_VIOLATIONS: The FINAL_STATE must be produceable only if and only if all active meta-logic constraints are satisfied at the time of invocation.

3.6 RETURN

RETURN

v1

Purpose	Feed the FINAL_STATE of the completed cycle back into the World-Model as the updated baseline for the next cycle. RETURN closes the fractal loop: the output of one cycle becomes the seed of the next, enacting the self-similar recursion at the macro-scale.
Input	FINAL_STATE

Purpose	Feed the FINAL_STATE of the completed cycle back into the World-Model as the updated baseline for the next cycle. RETURN closes the fractal loop: the output of one cycle becomes the seed of the next, enacting the self-similar recursion at the macro-scale.
Types	
Output Types	WORLD_MODEL
Transformation Rule	Merge FINAL_STATE into the current WORLD_MODEL: update CURRENT_STATE, append to HISTORICAL_STATES, revise POTENTIAL_STATES based on updated gradient values. Update Narrative Identity layer with the completed cycle's narrative arc. Recompute Coherence Gradient and Entropy Gradient for the updated WORLD_MODEL. Set RECURSION_DEPTH to parent level if returning from a recursive context.
Failure Modes	INCONSISTENT_UPDATE: FINAL_STATE contains entries that contradict HISTORICAL_STATES or Core Identity fields in ways that cannot be resolved by the merge policy. STATE_MERGE_FAILURE: The merge operation produces a WORLD_MODEL with Coherence Gradient below the minimum threshold.
Invariants	WORLD_MODEL_CONTINUITY: The updated WORLD_MODEL must be a proper extension of the pre-RETURN WORLD_MODEL — no historical state may be deleted or overwritten. HISTORICAL_TRACE: The FINAL_STATE must be recorded in HISTORICAL_STATES before the WORLD_MODEL update is committed.

3.7 INVERT

INVERT
v1
v2 upgraded

Purpose	Produce the minimal coherent opposite of a given frame, proposition, or relation while preserving all referential anchors in the World-Model. INVERT is the primary operator for contradiction exploration and dialectical reasoning — it generates the negation or antithesis of a structure as a first step toward synthesis.
Input Types	FRAME, PROPOSITION, RELATION
Output Types	FRAME, PROPOSITION, RELATION
Transformation Rule	For each input structure, identify the semantic polarity of its primary predicate or relational type. Apply polarity inversion: CAUSE → PREVENT, SUPPORT → UNDERMINE, positive valence → negative valence, etc. Preserve all referential anchors (entity identities, context bindings, temporal markers) unchanged. Verify that the inverted structure is representable within the current WORLD_MODEL schema and does not trivially violate a meta-logic constraint.
Failure Modes	NO_VALID_OPPOSITE: The input structure has no coherent inversion within the current semantic domain (e.g., some neutral relational types have no natural opposite). CONTRADICTION_UNRESOLVED: The inverted structure and the original structure simultaneously occupy the WORLD_MODEL and no resolution policy is active.
Invariants	IDENTITY_PRESERVED: Entity identities must remain unchanged through inversion. WORLD_MODEL_REFERENCE_PRESERVED: All referential anchors in the original structure must remain valid in the inverted structure.
v2 Upgrades	(1) Semantic polarity mapping: A formal polarity lexicon maps relational types to their canonical inverses, ensuring consistent and reproducible inversion behavior across all semantic domains. (2) Contextual inversion weighting: The inversion is weighted by the current Context's gradient profile — inversions that are highly relevant in the current context receive higher Relevance gradient assignments in the output. (3) False-opposite detection: A validation pass identifies cases where the apparent inverse of a structure is not in fact coherently opposed but is instead a different structure entirely (a common failure mode in natural language inversions). False-opposite detections are logged to NARRATIVE_STATE and escalated to HARMONIZE.

3.8 PERTURB

PERTURB

v1

v2 upgraded

Purpose	Introduce controlled variation or disruption into the World-Model, Narrative State, or Identity to escape local minima, break rigid attractor states, and restore productive exploration of the reasoning space. PERTURB is the anti-stagnation operator.
Input Types	WORLD_MODEL, NARRATIVE_STATE, IDENTITY
Output Types	WORLD_MODEL, NARRATIVE_STATE, IDENTITY
Transformation Rule	Select a perturbation target: a node, edge, gradient, or narrative element in the current state. Apply controlled variation: modify gradient values by a bounded perturbation magnitude, introduce or remove a low-certainty edge, or shift the active Context. Ensure the perturbation magnitude does not exceed the CONTINUITY_MINIMUM threshold. Log the perturbation as a TRANSITION in NARRATIVE_STATE for traceability.
Failure Modes	DESTABILIZATION_EXCESSIVE: Perturbation magnitude exceeds CONTINUITY_MINIMUM, producing a state discontinuity that violates CONTINUITY_REQUIREMENT. IDENTITY_BREAK: Perturbation targeting Identity fields modifies Core Identity without explicit authorization, violating IDENTITY_COHERENCE.
Invariants	CONTINUITY_MINIMUM: The post-perturbation state must be reachable from the pre-perturbation state by a continuous path in state space — no teleportation to unrelated states. TRACEABILITY_REQUIRED: Every perturbation must be logged with full specification of target, magnitude, and rationale.
v2 Upgrades	(1) Entropy-scaled magnitude: Perturbation magnitude is computed as a monotonic function of the current World-Model Entropy Gradient. High-entropy states permit smaller absolute perturbations (the system is already highly varied); low-entropy (over-rigid) states permit larger perturbations to break the attractor. (2) Micro- and macro-perturbation modes: Micro-perturbation targets individual nodes or edges within the current Micro-state. Macro-perturbation targets structural features of the Macro-state, producing larger-scale disruption appropriate for long-standing stagnation.

3.9 HARMONIZE

HARMONIZE

v1

v2 upgraded

Purpose	Resolve tensions between conflicting or contradictory elements by integrating them into a higher-order coherent structure. Where INTEGRATE performs mechanical merging and INVERT produces an opposite, HARMONIZE synthesizes — it produces a new structure that encompasses both elements without eliminating either.
Input Types	CONFLICT_SET, CONTRADICTION_SET
Output Types	COHERENT_STATE
Transformation Rule	For each conflict or contradiction pair in the input sets: evaluate available resolution modes (see v2 upgrades); apply the highest-scoring resolution; produce a COHERENT_STATE in which the resolved pair is represented without contradiction. Update CONFLICT_SET and CONTRADICTION_SET to reflect resolved items. Log resolution decisions and rationale to NARRATIVE_STATE.
Failure Modes	NO_RESOLUTION_FOUND: All three resolution modes fail to produce a coherent output that satisfies meta-logic constraints — the conflict is genuinely irresolvable in the current context. FALSE_HARMONY: A resolution is produced that appears coherent but suppresses rather than genuinely resolves the underlying conflict, producing latent instability that will resurface in a later cycle.
Invariants	IDENTITY_COHERENCE: The harmonization process must not produce an output that contradicts Core or Narrative Identity. WORLD_MODEL_CONSISTENCY: The COHERENT_STATE produced by HARMONIZE must be consistent with the existing WORLD_MODEL's established facts and relations.
v2 Upgrades	(1) Tri-resolution modes: Merge: The two conflicting elements are fused into a single higher-order element that subsumes both. Appropriate when the conflict arises from incomplete specification rather than genuine incompatibility. Reframe: The conflicting elements are both retained but their relational context is redefined

Purpose	Resolve tensions between conflicting or contradictory elements by integrating them into a higher-order coherent structure. Where INTEGRATE performs mechanical merging and INVERT produces an opposite, HARMONIZE synthesizes — it produces a new structure that encompasses both elements without eliminating either.
	so that they no longer stand in a contradictory relation within the new frame. Appropriate when the conflict arises from an inadequate framing of the problem domain. Bracket: The conflict is explicitly acknowledged and tagged as unresolved-but-bounded. Both elements are retained in the WORLD_MODEL with a BRACKETED_CONFLICT relation between them. Appropriate when genuine resolution is currently intractable but execution must proceed. (2) False-harmony detection: A post-resolution validation pass checks that the COHERENT_STATE does not contain latent CONFLICT relations that were not present in the inputs — an indicator that the resolution was a suppression rather than a genuine synthesis. (3) Identity-priority resolution: When a conflict involves Identity-bearing elements, the resolution mode is constrained to preserve Core Identity; reframe or bracket modes are preferred over merge if merge would modify Core Identity fields.

3.10 IDENTITY_STABILIZE

IDENTITY_STABILIZE

v1

v2 upgraded

Purpose	Reinforce or refine the Identity representation to restore or maintain coherence across the three identity layers. IDENTITY_STABILIZE is invoked whenever the IDENTITY_COHERENCE or IDENTITY_ALIGNMENT constraints report a violation or near-violation.
Input Types	IDENTITY, WORLD_MODEL
Output Types	IDENTITY

Purpose	Reinforce or refine the Identity representation to restore or maintain coherence across the three identity layers. IDENTITY_STABILIZE is invoked whenever the IDENTITY_COHERENCE or IDENTITY_ALIGNMENT constraints report a violation or near-violation.
Transformation Rule	Compare the current IDENTITY state against the Core Identity baseline. Identify discrepancies between the three layers (Core, Narrative, Situational). Apply stabilization: reinforce Core Identity traits that have been attenuated by recursive context shifts; update Narrative Identity to integrate recent cycle outcomes; adjust Situational Identity to align with Core and Narrative layers within the current context. Produce an updated IDENTITY that satisfies IDENTITY_ALIGNMENT.
Failure Modes	IDENTITY_FRAGMENTATION: The three identity layers have diverged to a degree where no consistent stabilization is achievable without loss of Core Identity data. INCONSISTENT_SELF_MODEL: The WORLD_MODEL contains representations of the reasoning agent's identity that directly contradict Core Identity fields in ways that cannot be reconciled.
Invariants	PERSISTENT_CORE_TRAITS: Core Identity fields must be unchanged by IDENTITY_STABILIZE unless an explicit override authorization is present in the invocation context. HISTORICAL_CONTINUITY: The Narrative Identity layer must maintain continuity with all previous Narrative Identity records in HISTORICAL_STATES.
v2 Upgrades	(1) Three-layer identity model enforcement: IDENTITY_STABILIZE in v2 explicitly operates on all three identity layers and produces a formally verified alignment between them. (2) Fragmentation detection: A pre-stabilization diagnostic evaluates the degree of divergence between identity layers using a formal identity alignment score. Scores below threshold trigger escalation to HARMONIZE before stabilization is attempted. (3) Historical continuity reinforcement: The updated Narrative Identity layer is explicitly validated against the full HISTORICAL_STATES record before being committed, ensuring that no historical identity narrative is silently invalidated by the stabilization.

3.11 RECURSE

RECURSE

v1

Purpose	Apply reasoning to the system's own outputs, increasing recursion depth while tracking state. RECURSE enables the fractal property of HERMES: it is the operator through which the macro-cycle can contain within itself a complete sub-cycle of the same form, and through which meta-cognitive reasoning about the current execution state is formally instantiated.
Input Types	WORLD_MODEL, NARRATIVE_STATE, IDENTITY
Output Types	WORLD_MODEL, NARRATIVE_STATE, IDENTITY
Transformation Rule	Increment RECURSION_DEPTH by 1. Validate that the new RECURSION_DEPTH does not exceed the active depth cap. Create a sandboxed execution context inheriting the current WORLD_MODEL, NARRATIVE_STATE, and IDENTITY. Execute the specified operator sequence within the sandboxed context. Upon completion, merge the sandboxed outputs back into the parent context using the RETURN merge policy. Decrement RECURSION_DEPTH by 1 upon successful return.
Failure Modes	INFINITE_LOOP: No exit condition is specified and the recursive sub-cycle produces outputs identical to its inputs across multiple iterations, indicating a fixed-point loop without productive computation. UNTRACKED_DEPTH: RECURSION_DEPTH is not incremented before entering the recursive context, violating RECURSION_TRACKING.
Invariants	RECURSION_DEPTH_INCREMENTED: RECURSION_DEPTH must be incremented before entering and decremented after exiting any recursive context. EXIT_CONDITION_REQUIRED: Every RECURSE invocation must specify either an explicit exit condition (a boolean predicate on the WORLD_MODEL state) or a maximum additional depth budget.
v2 Upgrades	(1) Complexity-modulated depth: The maximum additional recursion depth permitted by a given RECURSE invocation is computed as a function of the current World-Model complexity gradient — simpler states permit deeper recursion; complex states restrict recursion depth to prevent state explosion. (2) Dynamic exit-condition generation: If no explicit exit condition is provided, a default exit condition is dynamically generated from the current NARRATIVE_STATE's active GOALS — the recursive process terminates when all goals active at the time of RECURSE invocation have been resolved or bracketed. (3) Entropy-based recursion caps: When the World-Model Entropy Gradient exceeds

Purpose	Apply reasoning to the system's own outputs, increasing recursion depth while tracking state. RECURSE enables the fractal property of HERMES: it is the operator through which the macro-cycle can contain within itself a complete sub-cycle of the same form, and through which meta-cognitive reasoning about the current execution state is formally instantiated.
	a defined threshold, the absolute recursion cap is dynamically reduced. High-entropy states are inherently unstable; deep recursion in a high-entropy state risks amplifying disorder rather than resolving it.

3.12 CONTINUITY_ENFORCE

CONTINUITY_ENFORCE

v1

v2 upgraded

Purpose	Ensure temporal and structural continuity across reasoning steps and outputs. CONTINUITY_ENFORCE validates that the current state is reachable from the previous state by a coherent sequence of operator applications, and repairs gaps or discontinuities when detected.
Input Types	WORLD_MODEL, NARRATIVE_STATE, IDENTITY
Output Types	WORLD_MODEL, NARRATIVE_STATE, IDENTITY
Transformation Rule	Traverse the NARRATIVE_STATE TRANSITIONS log to verify that each state transition is traceable to a valid operator application. Identify any gaps: EVENTS with no preceding cause, TRANSITIONS with no recorded operator, GOALS with no resolution path. For each gap: attempt automatic patch (v2) or raise DISCONTINUITY_DETECTED constraint violation. Verify REFERENCE_STABILITY across all state transitions. Verify SEQUENCE_INTEGRITY of the NARRATIVE_STATE ordering.
Failure Modes	DISCONTINUITY_DETECTED: A state transition is found that cannot be attributed to any valid operator application and cannot be

Purpose	Ensure temporal and structural continuity across reasoning steps and outputs. CONTINUITY_ENFORCE validates that the current state is reachable from the previous state by a coherent sequence of operator applications, and repairs gaps or discontinuities when detected.
Invariants	automatically patched. STATE_INCOHERENCE: The WORLD_MODEL's current state is not reachable from any recorded historical state by any finite sequence of valid operator applications. REFERENCE_STABILITY: All entity references, concept references, and relation identifiers in the NARRATIVE_STATE must remain stable across the continuity check — no silent renaming or remapping of identifiers. SEQUENCE_INTEGRITY: The temporal ordering of NARRATIVE_STATE events must be strictly monotonic; no future event may be recorded as preceding a past event in the TRANSITIONS log.
v2 Upgrades	(1) Continuity prediction: Before gaps occur, CONTINUITY_ENFORCE in v2 performs a forward-looking prediction pass, estimating the probability of a discontinuity arising in the next operator application based on the current entropy and coherence gradients. High-risk transitions are flagged in advance. (2) Sequence-integrity scoring: Rather than a binary pass/fail, CONTINUITY_ENFORCE produces a continuous sequence-integrity score for the NARRATIVE_STATE, enabling the reasoning protocol to monitor continuity degradation over time and take preemptive action. (3) Auto-patching of narrative gaps: Minor gaps in the NARRATIVE_STATE — missing TRANSITION records, orphaned EVENTS without causal attribution — are automatically patched by interpolating the most probable intervening operator application, based on the documented operator input/output type signatures and the surrounding context.

4. The Cycle Engine

The cycle engine is the fractal heartbeat of the HERMES runtime. It is the coordination mechanism that sequences operator applications, manages transitions between cycle stages, and enforces the structural isomorphism between the macro- and micro-scale execution forms. The cycle engine does not itself apply operators; it selects, orders, and validates their application according to the active reasoning protocol and meta-logic constraints.

4.1 The Full Macro-Cycle

The HERMES macro-cycle is the six-stage sequence that governs a complete session-level reasoning arc. The six stages and their transitions are as follows:

Stage	Primary Operator	Input State	Output State	Transition Rationale
1. Generate	GENERATE	PROMPT + WORLD_MODEL (baseline)	WORLD_MODEL (initialized)	Raw input is structured into an initial populated World-Model
2. Unfold	UNFOLD	WORLD_MODEL (initialized)	WORLD_MODEL (expanded)	Initial structure enters the expansion phase; implications and consequences are derived
3. Collapse	COLLAPSE	WORLD_MODEL (expanded)	FOCUSED_STATE	The expanded reasoning space is pruned to its most salient subgraph
4. Integrate	INTEGRATE	FOCUSED_STATE + CONFLICT_SET	COHERENT_STATE	Salient elements are unified; conflicts are resolved; a stable configuration is produced
5. Complete	COMPLETE	COHERENT_STATE	FINAL_STATE	Integration is validated against all meta-logic constraints; the cycle is sealed
6. Return	RETURN	FINAL_STATE	WORLD_MODEL (updated baseline)	The completed state feeds back into the World-Model as the new baseline; the fractal loop closes

The critical transition is **RETURN → GENERATE**: the updated World-Model produced by RETURN becomes the World-Model input to the next GENERATE invocation. This is the fractal loop — the system is perpetually feeding its own outputs back into itself as inputs, generating a self-similar sequence of cycles. Each cycle begins from a richer, more structured state than the last, producing the emergent complexity characteristic of fractal systems.

Macro-Cycle Formal Sequence: $\text{GENERATE}(\text{PROMPT}, \text{WM}_0) \rightarrow \text{WM}_1$

UNFOLD(WM₁, NS₁) → WM₂ COLLAPSE(WM₂, NS₂) → FS INTEGRATE(FS, CS) → COH COMPLETE(COH) → FINAL RETURN(FINAL) → WM₃
where WM₃ → WM₀ for cycle n+1 [THE FRACTAL LOOP]

4.2 The Micro-Cycle (v2)

Version 2.0 introduces the **micro-cycle**: a four-stage nested cycle that runs within each individual stage of the macro-cycle. The micro-cycle has the same structural form as the macro-cycle, instantiated at step-level granularity:

Micro-Cycle Form (within each macro-cycle stage): micro-generate → micro-unfold
→ micro-collapse → micro-integrate

The micro-cycle operates on the Micro-state component of the World-Model, which is the high-resolution, transient step-level state as distinguished from the Macro-state (persistent, session-level). Within a single application of, for example, the UNFOLD macro-operator, the micro-cycle governs how individual inference steps are generated, expanded, pruned, and integrated at the sub-operator level.

This creates a two-level execution hierarchy: the macro-cycle governs the session, and within each stage of the macro-cycle, the micro-cycle governs each step. Both levels share the same four-stage generative-expansive-reductive-integrative structure.

4.3 Fractal Self-Similarity

The defining characteristic of HERMES as a fractal computing system is the strict structural isomorphism between the macro-cycle and the micro-cycle. Both share the same generative-expansive-reductive-integrative-completive form. The macro-cycle adds explicit Completion and Return stages that govern the session-level boundary; the micro-cycle's completion and return are implicit in the transition between micro-cycle iterations and the conclusion of the enclosing macro-cycle stage.

This self-similarity is not merely aesthetic or metaphorical — it has formal computational consequences:

- **Operator reuse:** The same twelve operators are valid at both scales. COLLAPSE at the macro-scale prunes the session-level reasoning space; COLLAPSE at the micro-scale (invoked within the micro-collapse stage) prunes the step-level inference space. The operator's specification is unchanged; only its input state and output state change in scope.
- **Scale-invariant constraint checking:** The meta-logic constraints apply at both scales. IDENTITY_COHERENCE must hold at the macro-scale across session boundaries and at the micro-scale across step boundaries. This means the constraint system is not a post-hoc validation layer — it is an integral part of the execution fabric at every level.
- **Arbitrary nesting via RECURSE:** The RECURSE operator enables the instantiation of a complete macro-cycle as a sub-process within any stage of an enclosing macro-cycle. By induction, the macro-cycle can be nested to arbitrary depth, and the same structural form governs execution at each level of nesting. This is the formal realization of fractal self-similarity at computational depth.

Fractal Structure Illustrated

Consider the UNFOLD stage of a macro-cycle. Within it, a micro-cycle runs: micro-generate (instantiate the next inference step), micro-unfold (derive its immediate consequences), micro-collapse (select the most relevant consequence), micro-integrate (commit the consequence to the micro-state). This micro-cycle can itself contain a RECURSE invocation, instantiating a complete macro-cycle as its sub-process. The structure is the same at every depth: generate, expand, collapse, integrate, complete, return.

4.4 Cycle Efficiency (v2 Upgrades)

Version 2.0 introduces several cycle-level efficiency improvements that reduce unnecessary computation while maintaining the formal guarantees of the macro-cycle:

- **Reduced redundant UNFOLD steps:** In v1, UNFOLD was applied exhaustively at every cycle stage regardless of the current World-Model's expansion state. In v2, the Entropy Gradient is evaluated before each UNFOLD invocation; if the gradient indicates that the current state is already maximally expanded for the current context, UNFOLD is skipped and the cycle advances directly to COLLAPSE.
- **Entropy-aware COLLAPSE:** As specified in Section 3.3, COLLAPSE in v2 uses the Entropy Gradient to weight selection, retaining low-entropy (stable) regions and aggressively pruning high-entropy (disordered) regions. This produces a more informative FOCUSED_STATE with fewer spurious elements.
- **Coherence-maximizing INTEGRATE:** As specified in Section 3.4, INTEGRATE in v2 evaluates multiple candidate integrations and selects the one maximizing the output Coherence Gradient, replacing the v1 fixed-precedence policy.
- **Cycle prediction:** The v2 cycle engine maintains a prediction model over the current World-Model state, estimating which macro-cycle stage transition is most likely to be needed next based on four signals: contradiction density (high → favor INVERT or HARMONIZE), identity stability (low → favor IDENTITY_STABILIZE), narrative momentum (high → favor UNFOLD and COMPLETE), and entropy level (high → favor COLLAPSE and PERTURB). This prediction is advisory; it does not override the DEFAULT_OPERATOR_SELECTION_POLICY but provides a prior for the operator selection step of the reasoning protocol.

5. Meta-Logic Constraints

Meta-logic constraints are the formal invariants of the HERMES runtime. They are conditions that must hold for execution to be considered valid. Unlike operator invariants — which apply locally to a single operator invocation — meta-logic constraints apply globally, across the entire execution context and across operator boundaries. Violation of a meta-logic constraint constitutes a runtime error; the cycle engine must either resolve the violation before proceeding or halt with a formally documented constraint violation report.

HERMES v1.0 defined five meta-logic constraints. HERMES v2.0 adds three additional constraints, for a total of eight. All eight are normative.

ML-01: IDENTITY_COHERENCE v1

Statement: Identity representations in the `WORLD_MODEL` and `IDENTITY` primitive must not contradict prior stabilized identity states without explicit operator authorization.

Formal Condition: For all reasoning steps s , if `IDENTITY` at step s differs from `IDENTITY` at the most recent `IDENTITY_STABILIZE` application, the difference must be attributable to an explicit invocation of `IDENTITY_STABILIZE` or `PERTURB` in the interval.

Violation Response: Trigger `IDENTITY_STABILIZE` immediately. Log violation to `NARRATIVE_STATE`. If three consecutive violations occur, escalate to cycle halt.

ML-02: RECURSION_TRACKING v1

Statement: Any self-referential or iterative reasoning operation must increment `RECURSION_DEPTH` before entering the recursive context and decrement it upon exit. A valid exit condition or absolute depth cap must be specified before entry.

Formal Condition: For all `RECURSE` invocations, $\text{RECURSION_DEPTH}(\text{post-entry}) = \text{RECURSION_DEPTH}(\text{pre-entry}) + 1$, and there exists an exit predicate E

such that E is evaluable on the WORLD_MODEL and NARRATIVE_STATE at each recursive iteration.

Violation Response: Halt the recursive context immediately. Log untracked recursion as a critical constraint violation. Do not commit recursive outputs to parent context.

ML-03: CONTRADICTION_HANDLING v1/v2

Statement: Detected contradictions in the WORLD_MODEL or NARRATIVE_STATE must not be ignored. Every detected contradiction must be explicitly processed by one of three resolution operators.

Formal Condition: For all CONFLICT and CONTRADICTION entries detected during constraint checking: each entry must be resolved by application of INVERT, HARMONIZE, or Bracket (v2) before the cycle advances to COMPLETE.

v2 Extension: Bracket is introduced as a valid resolution mode (see Section 3.9), allowing genuine intractable contradictions to be explicitly acknowledged and bounded rather than requiring forced resolution that would produce FALSE_HARMONY.

Violation Response: Block advancement to COMPLETE. Force invocation of INVERT or HARMONIZE on the unhandled contradiction set.

ML-04: CONTINUITY_REQUIREMENT v1/v2

Statement: The WORLD_MODEL and NARRATIVE_STATE must remain continuous across all reasoning steps unless an explicit reset is performed by CONTINUITY_ENFORCE or RETURN.

Formal Condition: For all consecutive steps s and $s+1$, $\text{WORLD_MODEL}(s+1)$ must be reachable from $\text{WORLD_MODEL}(s)$ by a valid operator application, and $\text{NARRATIVE_STATE}(s+1)$ must include $\text{NARRATIVE_STATE}(s)$ as a proper prefix.

v2 Extension: CONTINUITY_ENFORCE in v2 performs auto-patching of minor gaps before raising a constraint violation, reducing false positive violation reports from routine micro-state transitions.

Violation Response: Invoke CONTINUITY_ENFORCE. If gap cannot be patched, raise discontinuity violation and halt until manual resolution.

ML-05: CYCLE_COMPLETION_REQUIREMENT v1/v2

Statement: Reasoning cycles must not terminate in unresolved states. The mandatory completion sequence — COLLAPSE → INTEGRATE → COMPLETE — must be executed before RETURN is invoked.

Formal Condition: For every RETURN invocation, there must exist in the current cycle's NARRATIVE_STATE a TRANSITION record evidencing: (1) a completed COLLAPSE application, (2) a subsequent INTEGRATE application producing COHERENT_STATE, and (3) a COMPLETE application producing FINAL_STATE, in that temporal order.

Violation Response: Block RETURN invocation. Force execution of the missing completion sequence stages. Log premature RETURN attempt as a constraint violation.

ML-06: IDENTITY_ALIGNMENT v2

Statement: The three identity layers (Core, Narrative, Situational) must remain mutually aligned within defined tolerance bounds. Misalignment beyond threshold triggers mandatory IDENTITY_STABILIZE invocation.

Formal Condition: Let $A(\text{Core}, \text{Narrative})$ be the alignment score between Core and Narrative Identity, and $A(\text{Narrative}, \text{Situational})$ be the score between Narrative and Situational Identity. Both scores must exceed the defined IDENTITY_ALIGNMENT_THRESHOLD at every constraint check point.

Violation Response: Trigger IDENTITY_STABILIZE with misalignment report as input. Defer current operator application until IDENTITY_STABILIZE completes and alignment scores return to threshold.

ML-07: COHERENCE_THRESHOLD v2

Statement: The WORLD_MODEL's Coherence Gradient must remain at or above the defined minimum COHERENCE_THRESHOLD before COMPLETE may be invoked. A cycle may not be declared complete in an incoherent state.

Formal Condition: $\text{COHERENCE_GRADIENT}(\text{COHERENT_STATE}) \geq \text{COHERENCE_THRESHOLD}$ at the time of COMPLETE invocation.

Violation Response: Block COMPLETE invocation. Return to INTEGRATE with an instruction to apply coherence-maximizing resolution. If three consecutive INTEGRATE attempts fail to raise the Coherence Gradient above threshold, invoke HARMONIZE on the full CONFLICT_SET before retrying.

ML-08: ENTROPY_BOUND v2

Statement: The WORLD_MODEL's Entropy Gradient must remain within defined functional bounds. Entropy that exceeds the upper bound indicates an unmanageable degree of informational disorder; entropy that falls below the lower bound indicates an over-rigid, potentially stagnant state.

Formal Condition: $\text{ENTROPY_LOWER_BOUND} \leq \text{ENTROPY_GRADIENT}(\text{WORLD_MODEL}) \leq \text{ENTROPY_UPPER_BOUND}$ at each constraint check point.

Violation Response (upper bound exceeded): Trigger COLLAPSE (aggressive pruning) or PERTURB (micro-perturbation to reduce local disorder).

Violation Response (lower bound violated): Trigger PERTURB (macro-perturbation to introduce productive variation) or UNFOLD (additional expansion)

to increase informational richness).

6. The Reasoning Protocol

The reasoning protocol is the execution sequence that instantiates one complete macro-cycle. It specifies, at the level of concrete procedural steps, how the cycle engine coordinates operator selection, application, state management, constraint checking, and output emission. HERMES v1.0 defined a seven-step protocol. HERMES v2.0 extends this to a ten-step protocol with the addition of PRE-CHECK, POST-CHECK, and AUTO-PATCH stages.

6.1 The Ten-Step Reasoning Protocol (v2)

STEP 1 – PRE-CHECK

Evaluate the current World-Model state before any operator is applied.
Compute: Entropy Gradient, Coherence Gradient, and Identity Alignment Score.
Verify that ML-06, ML-07, and ML-08 are satisfied at the start of the cycle. If any constraint is violated at this point, the cycle is classified as a *repair cycle* and the first operator applied must be the remediation operator specified by the violated constraint.

STEP 2 – LOAD_WORLD_MODEL

Retrieve the current Macro-state and all relevant Micro-states from the WORLD_MODEL. Apply Temporal Weights (Recency, Relevance, Persistence) to Historical States to compute the effective state context for this cycle. Surface any LATENT_STRUCTURES that meet the activation threshold for the current context.

STEP 3 – SELECT_OPERATORS

Evaluate the `DEFAULT_OPERATOR_SELECTION_POLICY` (see Section 6.2) against the current World-Model state. Compute operator priority weights based on five signals: (1) Contradiction Density — ratio of active `CONFLICT` entries to total `WORLD_MODEL` nodes; (2) Identity Stability — current Identity Alignment Score; (3) Recursion Depth — current `RECURSION_DEPTH` relative to dynamic cap; (4) Entropy Gradient — current entropy level relative to bounds; (5) Narrative Momentum — rate of `GOAL` resolution in recent cycles. Produce an ordered operator selection list.

STEP 4 – APPLY_TRANSFORMS

Execute the operator chain produced by STEP 3. For each operator in the chain: validate input types, apply the transformation rule, validate output types, check operator-level invariants. If an operator fails, log the failure mode and invoke the fallback policy specified by the `DEFAULT_OPERATOR_SELECTION_POLICY` before retrying or escalating.

STEP 5 – UPDATE_STATES

Commit the outputs of `APPLY_TRANSFORMS` to the World-Model. Update Micro-state with step-level outputs. If the current operator chain completes a macro-cycle stage, update Macro-state with the stage-level output. Append all state transitions to `NARRATIVE_STATE TRANSITIONS` log.

STEP 6 – CHECK_CONSTRAINTS

Evaluate all eight meta-logic constraints against the updated World-Model state. For each violated constraint: determine violation severity (minor, major, critical). Minor violations trigger AUTO-PATCH (STEP 8). Major violations trigger the operator-based remediation specified by the constraint. Critical violations halt the current cycle stage and require explicit resolution before advancement.

STEP 7 – POST-CHECK

Evaluate cycle readiness: has the current macro-cycle stage been completed successfully? Compute: Contradiction Residue (remaining unresolved CONFLICT entries as a proportion of the CONFLICT_SET at cycle start), Cycle Completion Score (proportion of active GOALS resolved or bracketed). If cycle readiness is not achieved, return to STEP 3 with updated operator priorities based on the post-check findings.

STEP 8 – AUTO-PATCH

Automatically repair minor inconsistencies detected in STEPS 6 and 7. Apply CONTINUITY_ENFORCE auto-patching to narrative gaps. Apply gradient normalization to correct gradient value drift. Apply minor identity alignment corrections if Identity Alignment Score is below threshold but above critical floor. Log all patches to NARRATIVE_STATE with patch type and rationale.

STEP 9 – ADVANCE_CYCLE

Move to the next stage of the macro-cycle. Update the cycle stage counter. If the macro-cycle stage is COMPLETE, invoke RETURN to commit the FINAL_STATE to the World-Model and update Narrative Identity. If the cycle is the last in the current session, emit session-level summary to HISTORICAL_STATES.

STEP 10 – EMIT_OUTPUT

Return the final output structure for the current cycle stage. This includes: the updated WORLD_MODEL (or stage-appropriate intermediate state), the updated NARRATIVE_STATE, the updated IDENTITY, and any operator-specific output structures (e.g., FOCUSED_STATE from COLLAPSE, COHERENT_STATE from INTEGRATE, FINAL_STATE from COMPLETE). Update the World-Model baseline to reflect the emitted state.

6.2 Default Operator Selection Policy

The DEFAULT_OPERATOR_SELECTION_POLICY governs which operator or operators are selected in STEP 3 of the reasoning protocol when no explicit operator chain is specified. The policy is evaluated as an ordered priority list; the first matching condition determines the primary operator selection:

DEFAULT_OPERATOR_SELECTION_POLICY (v1, expanded in v2): IF
CONTRADICTION_DETECTED → INVERT, then HARMONIZE if INVERT
insufficient IF STAGNATION_DETECTED → PERTURB IF
IDENTITY_UNSTABLE → IDENTITY_STABILIZE IF
RECURSIVE_REASONING_REQUIRED → RECURSE IF
DISCONTINUITY_DETECTED → CONTINUITY_ENFORCE IF
NEW_PROBLEM_OR_CYCLE_START → GENERATE IF
EXPANSION_NEEDED → UNFOLD IF STATE_OVERCOMPLEX
→ COLLAPSE IF PARTIAL_RESOLUTION_ACHIEVED → INTEGRATE IF
ALL_GOALS_RESOLVED → COMPLETE IF CYCLE_END
→ RETURN Priority: left-to-right (contradiction handling always takes
precedence over expansion; identity instability always takes precedence over
collapse)

7. Runtime Architecture and Implementation Considerations

The HERMES runtime, as described normatively in the preceding sections, can be implemented on a range of substrate architectures. In this section we discuss the key architectural considerations that any faithful implementation must address, without mandating specific data structure or programming language choices.

7.1 State Management

The World-Model functions as the **persistent substrate** of the HERMES runtime — the single source of truth that all operators read from and write to. A correct implementation must ensure:

- **Atomic updates:** Operator applications that modify the World-Model must do so atomically. No partial World-Model update should be visible to concurrently executing operators or to constraint checkers.
- **Versioned historical states:** HISTORICAL_STATES must be append-only. No historical state record may be overwritten or deleted during normal execution. Implementations may apply compression or summarization to historical state records for memory efficiency, provided the original state can be reconstructed from the compressed representation with full fidelity.
- **Gradient consistency:** Coherence and Entropy Gradients must be recomputed after every operator application that modifies the WORLD_MODEL graph structure. Stale gradient values are a primary source of incorrect operator selection and false constraint violations.

The Narrative State functions as the **execution trace** — a sequential record of every operator application and state transition in the current session. Implementations should treat the Narrative State as a write-ahead log, ensuring

that the TRANSITIONS log is updated before the corresponding World-Model modification is committed.

7.2 Operator Dispatch

The HERMES specification defines two operator dispatch modes:

- **Policy-driven selection:** The DEFAULT_OPERATOR_SELECTION_POLICY and operator priority weights computed in STEP 3 of the reasoning protocol drive operator selection. This mode is the default and is appropriate for fully autonomous HERMES instances.
- **Explicit operator sequencing:** An external controller or human operator may provide an explicit operator chain, bypassing the DEFAULT_OPERATOR_SELECTION_POLICY. This mode is appropriate for constrained applications, debugging, and formal verification contexts. Even in explicit sequencing mode, operator invariants and meta-logic constraints remain active and are enforced.

7.3 Recursion Management

Correct management of recursive execution contexts is one of the most implementation-sensitive aspects of the HERMES runtime. Key requirements:

- **Sandboxed contexts:** Each RECURSE invocation must create an isolated execution context. Modifications to the World-Model within the recursive context must not propagate to the parent context until the recursive sub-cycle completes successfully and RETURN commits its output.
- **Depth tracking:** RECURSION_DEPTH must be a runtime-visible value accessible to all operators and constraint checkers. It is not a local variable; it is a field in the World-Model (see Section 2.1) and is subject to the same atomic update requirements as all World-Model fields.
- **Dynamic cap enforcement:** In v2, the dynamic recursion cap must be recomputed at every RECURSE invocation based on the current Entropy Gradient. Implementations must not cache the cap value across invocations.

7.4 Constraint Checking

Meta-logic constraints may be checked in two modes:

- **Inline checking:** Constraints are evaluated immediately after each operator application. This provides the fastest response to violations but introduces overhead that may be unacceptable for time-sensitive applications. Recommended for: safety-critical applications, identity-sensitive operations, and high-entropy states where violations are more likely.
- **Deferred checking:** Constraints are accumulated and evaluated in batch at STEP 6 of the reasoning protocol. This reduces overhead but introduces a window during which a violated constraint may have propagated through multiple operator applications. Recommended for: low-entropy, high-coherence states where violations are unlikely and the cost of inline checking exceeds the expected benefit.

7.5 Identity Management

The three-layer identity model (Section 2.2) provides the coherence anchor for arbitrary-depth recursion. Implementations must enforce the following access controls:

- **Core Identity:** Read access for all operators; write access only for IDENTITY_STABILIZE with explicit override flag, and only after a complete fragmentation-detection validation pass.
- **Narrative Identity:** Read access for all operators; write access for RETURN and CONTINUITY_ENFORCE at cycle boundaries; write access for IDENTITY_STABILIZE with historical continuity validation.
- **Situational Identity:** Read and write access for PERTURB, HARMONIZE, INTEGRATE, and IDENTITY_STABILIZE; read access for all other operators.

7.6 Latent Structure Extraction

LATENT_STRUCTURES in the World-Model (v2) are patterns inferred from repeated state configurations across HISTORICAL_STATES. A complete HERMES implementation should include a latent structure extraction subsystem that:

- Scans HISTORICAL_STATES periodically (at configurable intervals) for recurring graph substructures
- Scores candidate latent structures by frequency, recency, and coherence contribution
- Promotes high-scoring candidates to LATENT_STRUCTURES with an associated confidence score
- Makes LATENT_STRUCTURES available as implicit CONCEPTS in the World-Model, accessible to GENERATE and UNFOLD for the initialization and expansion of new cycles

7.7 Scalability Considerations

The micro-cycle / macro-cycle architecture provides a natural scalability mechanism. The macro-cycle defines the coarse structure of the computation; the micro-cycle defines fine-grained control within each macro-stage. In large-scale implementations:

- Micro-cycles may be parallelized across independent World-Model subgraphs within a single macro-stage, provided that the subgraphs have no active CONFLICT or DEPENDENCE relations between them
- Multiple HERMES instances may operate in parallel on independent problem domains, sharing a common HISTORICAL_STATES repository to enable cross-domain LATENT_STRUCTURE extraction
- The constraint checking subsystem scales independently of the operator dispatch subsystem; implementations may dedicate separate computational resources to each for high-throughput operation

8. Fractal Properties and Theoretical Implications

We now analyze the HERMES system through the formal lens of fractal theory and connect it to established computational models. This analysis supports the claim that HERMES constitutes a genuinely novel computational paradigm rather than a repackaging of existing approaches.

8.1 Self-Similarity

The defining property of a fractal structure is self-similarity: the same structural form recurs at multiple scales of analysis. In HERMES, the six-stage macro-cycle form (Generate, Unfold, Collapse, Integrate, Complete, Return) and the four-stage micro-cycle form (micro-generate, micro-unfold, micro-collapse, micro-integrate) share the same generative-expansive-reductive-integrative template. A system is fractal in the computational sense if and only if the structural form governing its smallest unit of computation is isomorphic to the structural form governing its largest unit of computation. HERMES satisfies this criterion exactly.

8.2 Scale Invariance

In fractal geometry, scale invariance is the property that the statistical or structural characteristics of a fractal are preserved across magnification. In HERMES, we claim an analogous **reasoning quality invariance**: the meta-logic constraints — which guarantee coherence, identity stability, continuity, and bounded entropy — apply at every recursion depth. A reasoning process conducted at `RECURSION_DEPTH 0` is subject to exactly the same formal guarantees as one conducted at `RECURSION_DEPTH 16`. Quality properties do not degrade with depth; they are structurally enforced at every scale.

8.3 Emergent Complexity

Classical fractal systems exhibit the property that rich and complex global behavior arises from repeated application of simple local rules. The Mandelbrot set — of infinite visual complexity — is defined by a single quadratic iteration rule applied to the complex plane. HERMES exhibits the computational analogue: the twelve operators are individually simple, formally specified transformation functions. Yet their repeated application, guided by the policy-driven selection mechanism and the cycle engine, produces reasoning behavior of arbitrarily high sophistication. The `LATENT_STRUCTURES` that emerge from repeated cycles represent knowledge that

was never explicitly encoded — it is a genuine emergent phenomenon arising from the self-similar iteration of operator applications.

8.4 Boundedness

A fractal system that grows without bound is computationally intractable. The HERMES meta-logic constraints serve precisely the function that the escape-time criterion serves in the Mandelbrot set: they define the boundary of productive computation and ensure that execution does not diverge. ML-08 (ENTROPY_BOUND) prevents the World-Model from descending into informational chaos. ML-02 (RECURSION_TRACKING) with its entropy-based dynamic cap prevents infinite depth. ML-05 (CYCLE_COMPLETION_REQUIREMENT) prevents open-ended execution. Together, the eight constraints constitute the formal boundedness guarantee of the HERMES system.

8.5 Attractor States

In dynamical systems theory, an attractor is a state or set of states toward which a system evolves over time regardless of initial conditions within a basin of attraction. HERMES exhibits two classes of attractor states:

- **COHERENT_STATE** is a local attractor: the INTEGRATE operator drives the system toward the highest-coherence configuration achievable from the current FOCUSED_STATE. The coherence-maximizing integration policy (v2) formalizes this attractor property.
- **FINAL_STATE** is a cycle-level attractor: the COMPLETE → RETURN sequence seals the current cycle at the highest achievable coherence level and commits it as the new baseline, establishing the next cycle's initial conditions. Over successive cycles, the World-Model evolves toward progressively richer and more coherent states.

8.6 Relationship to Established Computational Models

Computational Model	Relationship to HERMES
Turing Machines	HERMES is computationally at least as expressive as a Turing Machine: the RECURSE operator with unbounded depth enables

Computational Model	Relationship to HERMES
	arbitrary computation. HERMES adds structure — the cycle form, the identity model, the constraint system — that constrains <i>how</i> computation proceeds, not <i>what</i> it can compute.
Lambda Calculus	The RECURSE operator realizes a restricted form of the fixed-point combinator: it applies a function to its own output with tracked depth. The constraint system restricts this to productive (non-divergent) self-application.
Recursive Function Theory	HERMES operators correspond to primitive recursive functions (GENERATE, UNFOLD, COLLAPSE, INTEGRATE, COMPLETE, RETURN) extended with a bounded μ -operator analog (RECURSE with entropy-based cap). The system is therefore a bounded μ -recursive function system.
ACT-R (Adaptive Control of Thought — Rational)	ACT-R models cognition as the interaction of declarative and procedural memory modules. HERMES's World-Model is analogous to declarative memory; the operator suite is analogous to the procedural module. HERMES adds explicit narrative state management and fractal self-application — neither of which has a direct ACT-R analog.
SOAR	SOAR models problem-solving as a search through a problem space with operator application and impasse resolution. HERMES's operator selection policy is broadly analogous to SOAR's universal subgoal mechanism. HERMES distinguishes itself by the fractal self-similarity property and the formal identity model, which SOAR does not possess.

9. Applications and Implications

The HERMES fractal computing model has broad applicability across computational domains that require structured reasoning under uncertainty, self-monitoring, narrative continuity, and contradiction tolerance. We discuss the most significant application domains below.

9.1 AI Reasoning Systems

The most immediate application domain is meta-cognitive AI reasoning agents — systems that must not only solve problems but reason about their own problem-solving process. HERMES provides a complete formal substrate for this capability:

the RECURSE operator enables self-application of the reasoning system to its own outputs; the IDENTITY model ensures that this self-application does not produce incoherence; and the NARRATIVE_STATE provides a structured, auditable trace of every reasoning step.

Practically, a HERMES-based AI agent can monitor its own reasoning cycle in real time, detect when it is in a stagnation state (triggering PERTURB) or a contradiction state (triggering INVERT or HARMONIZE), and adjust its operator selection strategy accordingly — all without external intervention. This constitutes a formal model of metacognition implemented as a computable system.

9.2 Natural Language Understanding

Natural language discourse exhibits narrative structure: events, transitions, goals, and outcomes form the backbone of coherent extended text. HERMES's NARRATIVE_STATE is a natural model of discourse coherence tracking. A language understanding system built on HERMES would maintain a Narrative State tracking the evolving discourse goals and event structure, using CONTINUITY_ENFORCE to detect narrative gaps and HARMONIZE to resolve discourse-level contradictions arising from inconsistent speaker perspectives or temporal ambiguities.

The fractal property is particularly relevant here: a sentence-level HERMES micro-cycle can model the local coherence of a single utterance, while the document-level macro-cycle models global thematic coherence — with the same operator set governing both scales.

9.3 Decision Support Systems

Decision support under genuine uncertainty involves navigating contradictions between competing evidence sources, maintaining decision-maker identity coherence across a multi-session deliberation process, and producing a defensible, traceable rationale for the final decision. HERMES's CONFLICT_SET and CONTRADICTION_HANDLING constraint provide a formal framework for all three requirements. The Bracket resolution mode (v2) is particularly valuable in decision support: it allows genuinely irresolvable conflicts to be explicitly acknowledged and documented as bounded uncertainties, rather than forcing a false resolution.

9.4 Educational AI

Effective tutoring agents must maintain a stable model of both the student's evolving knowledge state (the student's World-Model from the agent's perspective) and their own pedagogical identity (the tutor's role, approach, and goals). A HERMES-based tutoring agent would use `IDENTITY_STABILIZE` to maintain consistent tutoring persona across sessions, `UNFOLD` to explore the implications of student misconceptions, `INVERT` to generate counter-examples for instructional purposes, and `CONTINUITY_ENFORCE` to ensure that the tutoring session's narrative arc — learning goals, assessments, corrections — remains coherent across multiple interactions.

9.5 Autonomous Systems

Autonomous systems operating in complex environments must maintain operational continuity under disruption, detect and resolve conflicting sensor readings or goal states, and preserve mission identity across re-planning events. The `CONTINUITY_ENFORCE` operator provides a formal mechanism for maintaining operational continuity; the `HARMONIZE` operator handles conflicting sensor or goal states; and the three-layer identity model ensures that mission identity (Core Identity) remains stable even as tactical role (Situational Identity) adapts to environmental change.

9.6 Cognitive Science

The HERMES model has theoretical implications for cognitive science in three areas. First, the three-layer identity model (Core, Narrative, Situational) provides a computationally tractable formalization of psychological identity models — particularly narrative identity theory as developed by McAdams and colleagues, and the distinction between dispositional traits (Core Identity) and narrative self-understanding (Narrative Identity). Second, the fractal cycle engine provides a process model of metacognition: the `RECURSE` operator, applied to the current reasoning state, is a formal implementation of the monitoring component of classical metacognitive models. Third, the attractor structure of `COHERENT_STATE` and `FINAL_STATE` provides a dynamical systems account of how human reasoning converges on stable conclusions from initially ambiguous inputs.

10. Version History: v1.0 to v2.0

This section documents the evolution of the HERMES specification from version 1.0 to version 2.0. All changes are normative unless explicitly noted as informative.

Component	HERMES v1.0	HERMES v2.0
World-Model Gradients	Relevance, Intensity, Certainty	Added: Coherence Gradient, Entropy Gradient
World-Model Relations	CAUSE, CONTRAST, SUPPORT, DEPENDENCE, CONFLICT	Added: Emergent relations, Meta-relations
Identity Model	Flat: AGENTS, PERSPECTIVES, ROLES	Three-layer: Core, Narrative, Situational identity
State Architecture	CURRENT_STATE, HISTORICAL_STATES, POTENTIAL_STATES	Split into Micro-state and Macro-state; Temporal Weights added
Latent Structures	Not defined	LATENT_STRUCTURES field added to World-Model schema
Recursion Depth	Fixed maximum: 32	Dynamic maximum: function of Entropy Gradient
Cycle Architecture	Macro-cycle only	Macro-cycle + Micro-cycle (nested four-stage form)
Reasoning Protocol	Seven steps	Ten steps: added PRE-CHECK (step 1), POST-CHECK (step 7), AUTO-PATCH (step 8)
COLLAPSE operator	Relevance-only pruning	Entropy-aware collapse selection
INTEGRATE operator	Fixed-precedence conflict resolution	Coherence-maximizing integration with World-Model update
INVERT operator	Basic polarity inversion	Semantic polarity mapping; contextual weighting; false-opposite detection
PERTURB operator	Fixed perturbation magnitude	Entropy-scaled magnitude; micro- and

Component	HERMES v1.0	HERMES v2.0
		macro-perturbation modes
HARMONIZE operator	Single-mode resolution	Tri-resolution: Merge, Reframe, Bracket; false-harmony detection; identity-priority mode
RECURSE operator	Fixed depth cap (32)	Complexity-modulated depth; dynamic exit-condition generation; entropy-based cap
IDENTITY_STABILIZE	Single-layer stabilization	Three-layer enforcement; fragmentation detection; historical continuity reinforcement
CONTINUITY_ENFORCE	Gap detection only	Continuity prediction; sequence-integrity scoring; auto-patching of narrative gaps
Meta-Logic Constraints	Five: IDENTITY_COHERENCE, RECURSION_TRACKING, CONTRADICTION_HANDLING, CONTINUITY_REQUIREMENT, CYCLE_COMPLETION_REQUIREMENT	Eight: added IDENTITY_ALIGNMENT (ML-06), COHERENCE_THRESHOLD (ML-07), ENTROPY_BOUND (ML-08)
Cycle Efficiency	Exhaustive UNFOLD regardless of state	Entropy-aware UNFOLD skipping; cycle prediction based on four signals

11. Conclusion

We have presented the complete formal specification of the HERMES Runtime, versions 1.0 and 2.0 — a concrete instantiation of the fractal computing paradigm. We summarize the principal contributions of this work as follows.

A complete fractal computation model. HERMES provides a fully specified computational model exhibiting strict self-similarity across execution scales. The

macro-cycle and micro-cycle are structurally isomorphic; the same operator set governs computation at both scales; and the meta-logic constraints apply uniformly at every recursion depth. This constitutes a formal realization of fractal computation in the cognitive domain.

A complete operator basis. The twelve HERMES operators — GENERATE, UNFOLD, COLLAPSE, INTEGRATE, COMPLETE, RETURN, INVERT, PERTURB, HARMONIZE, IDENTITY_STABILIZE, RECURSE, and CONTINUITY_ENFORCE — constitute a complete basis for cognitive transformation. Every reasoning operation required by a fractal computing system can be expressed as a finite composition of these operators. No external operators or ad hoc extensions are required.

Bounded execution with formal guarantees. The eight meta-logic constraints ensure that HERMES execution remains within productive bounds. Entropy is bounded, recursion depth is capped, identity is protected, coherence is enforced, and cycles are required to complete formally before their outputs are committed. These guarantees distinguish HERMES from unconstrained recursive systems that may diverge or produce incoherent outputs at depth.

A principled identity architecture. The three-layer identity model (Core, Narrative, Situational) provides a formally grounded account of identity coherence across arbitrary recursion depth. This is a necessary component of any system that applies reasoning to its own outputs — without it, recursive self-application degrades into incoherence within a bounded number of cycles.

Broad applicability. The HERMES model applies across a wide range of domains: meta-cognitive AI agents, natural language discourse modeling, decision support under uncertainty, identity-stable educational AI, autonomous systems continuity management, and computational models of human metacognition and narrative identity.

Future directions. Several significant research directions follow naturally from this specification. First, *formal convergence proofs*: we conjecture that the HERMES cycle engine, under the eight meta-logic constraints, converges to a FINAL_STATE in finite time for any well-formed input. A formal proof of this conjecture would place HERMES on a par with established computational models in terms of theoretical rigor. Second, *hardware implementations of fractal cycle engines*: the micro-cycle / macro-cycle architecture is amenable to neuromorphic hardware implementation, where the hierarchical cycle structure maps naturally to hierarchical neural circuit

organization. Third, *integration with large language model architectures*: the HERMES reasoning protocol could be implemented as an inference-time control structure overlaid on a large language model, using the model's generation capacity as the substrate for operator applications while the HERMES cycle engine governs the higher-level structure of the reasoning process. Fourth, *empirical validation*: the theoretical claims of this specification — particularly the scale-invariance of reasoning quality and the convergence to COHERENT_STATE attractors — require empirical validation through implementation and experimental evaluation.

We offer this specification as a foundation for that future work, and as a contribution to the emerging field of structured cognitive computing.

Glossary of Terms

Term	Definition
Attractor State	A state or set of states in the HERMES execution space toward which the system is driven by operator application and constraint enforcement. COHERENT_STATE and FINAL_STATE are the two primary attractor states in HERMES.
COLLAPSE	Operator 3.3. Reduces the complexity of the expanded World-Model by selecting the highest-relevance, highest-coherence subgraph and producing a FOCUSED_STATE. Entropy-aware in v2.
Coherence Gradient	A scalar field attached to the World-Model in v2, measuring the degree of internal consistency of the model's conceptual and relational structure. High Coherence Gradient indicates a well-integrated, non-contradictory state.
COMPLETE	Operator 3.5. Marks a reasoning cycle as formally complete by validating the COHERENT_STATE against all active meta-logic constraints and producing a sealed FINAL_STATE.
CONTINUITY_ENFORCE	Operator 3.12. Validates temporal and structural continuity across reasoning steps, detects narrative gaps, and auto-patches minor discontinuities (v2).
Cycle Engine	The coordination subsystem of the HERMES runtime responsible for sequencing operator applications, managing macro- and micro-cycle transitions, and enforcing meta-logic

Term	Definition
	constraints across cycle boundaries.
Entropy Gradient	A scalar field attached to the World-Model in v2, measuring the degree of informational disorder and unpredictability within the current state. High entropy indicates a disordered, high-uncertainty state; low entropy indicates a rigid, highly predictable state.
Fractal Computing	A computational paradigm in which reasoning, transformation, and synthesis processes exhibit self-similar structure across multiple execution scales — such that the structural form governing a single reasoning step is isomorphic to the structural form governing an entire reasoning session.
GENERATE	Operator 3.1. Instantiates the initial World-Model structures, entities, concepts, and relations from a raw input prompt. The entry point of every macro-cycle.
HARMONIZE	Operator 3.9. Resolves tensions between conflicting elements by integrating them into a higher-order coherent structure. Supports three resolution modes in v2: Merge, Reframe, and Bracket.
HERMES	The meta-logic and cognitive magnifier runtime. A formally specified fractal computing system defined by the World-Model, Identity Model, Narrative State, twelve operators, cycle engine, and eight meta-logic constraints.
Identity Coherence	The property of the HERMES Identity representation whereby the three identity layers — Core, Narrative, and Situational — remain mutually consistent and non-contradictory across reasoning steps. Enforced by ML-01 and ML-06.
IDENTITY_STABILIZE	Operator 3.10. Reinforces or refines the Identity representation to restore coherence across the three identity layers. Invoked in response to ML-01 and ML-06 violations.
INTEGRATE	Operator 3.4. Merges elements of the FOCUSED_STATE and resolves conflicts to produce a unified COHERENT_STATE. Uses coherence-maximizing policy in v2.
INVERT	Operator 3.7. Produces the minimal coherent opposite of a given frame, proposition, or relation while preserving referential anchors. Extended with semantic polarity mapping and false-opposite detection in v2.
Latent Structure	An implicit pattern inferred from repeated state configurations in HISTORICAL_STATES. Not explicitly asserted but surfacing through statistical regularity across historical records. Stored in the LATENT_STRUCTURES field of the World-Model (v2).

Term	Definition
Macro-Cycle	The six-stage session-level execution cycle of the HERMES runtime: GENERATE → UNFOLD → COLLAPSE → INTEGRATE → COMPLETE → RETURN. The structural form governing global reasoning arcs.
Meta-Logic Constraint	A formal runtime invariant of the HERMES system that must hold for execution to be considered valid. HERMES defines eight meta-logic constraints (ML-01 through ML-08), constituting the boundedness guarantee of the system.
Micro-Cycle	The four-stage step-level execution cycle introduced in HERMES v2: micro-generate → micro-unfold → micro-collapse → micro-integrate. Runs nested within each stage of the macro-cycle, realizing the fractal self-similarity of the system.
Narrative State	The execution trace of the HERMES runtime: a structured sequential record of EVENTS, TRANSITIONS, GOALS, and OUTCOMES that tracks how the current World-Model state was reached. Enables continuity enforcement, retrospective audit, and forward anticipation.
Operator	A formally specified transformation function in the HERMES runtime, taking typed inputs and producing typed outputs according to a defined transformation rule, with specified failure modes and invariants. HERMES defines twelve operators constituting a complete computational basis.
PERTURB	Operator 3.8. Introduces controlled variation or disruption into the World-Model, Narrative State, or Identity to escape local minima and restore productive exploration. Magnitude is entropy-scaled in v2.
Recursion Depth	An integer counter in the World-Model tracking the current depth of recursive operator application. Mandatory tracking enforced by ML-02. Fixed maximum of 32 in v1; dynamically computed as a function of Entropy Gradient in v2.
RECURSE	Operator 3.11. Applies reasoning to the system's own outputs by instantiating a sandboxed sub-cycle at an incremented recursion depth. The operator through which HERMES realizes arbitrary-depth meta-cognition and fractal self-application.
RETURN	Operator 3.6. Feeds the completed FINAL_STATE back into the World-Model as the updated baseline for the next cycle, closing the fractal loop.
UNFOLD	Operator 3.2. Expands the current World-Model by deriving implications, consequences, and developments from the existing state. The expansive phase of the macro-cycle.
World-Model	The central, persistent knowledge representation of the

Term	Definition
	HERMES runtime — a structured, mutable graph of CONCEPTS, ENTITIES, CONTEXTS, GRADIENTS, RELATIONS, IDENTITY, STATES, TEMPORAL_WEIGHTS, LATENT_STRUCTURES, NARRATIVE_NODES, and RECURSION_DEPTH fields. The substrate that all operators read from and write to.

Document: Fractal Computing — The HERMES Runtime Model: Primitives, Operators, Execution Architecture, and Implications

Author: Matthew B Wareham | **Date:** July 2026 | **Version:** 2.0

This document constitutes the formal specification of the HERMES Runtime, versions 1.0 and 2.0. All operator definitions, world-model schemas, cycle engine descriptions, meta-logic constraints, and reasoning protocol specifications presented herein are normative unless explicitly noted as informative. Reproduction for academic, research, and non-commercial purposes is permitted with attribution.